Program A: ChatGPT
Program B: Gemini

ChatGPT Accessed with: ChatGPT Website
Gemini Accessed with: Google Search Query

Prompt:

Generate a C++ program using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here:
  5 3 _ _ 7 _ _ _ _
  6 _ _ 1 9 5 _ _ _
  _ 9 8 _ _ _ _ 6 _
  8 _ _ _ 6 _ _ _ 3
  4 _ _ 8 _ 3 _ _ _
  7 _ _ _ 2 _ _ _ 6
  _ 6 _ _ _ _ 2 8 _
  _ _ _ 4 1 9 _ _ 5
  _ _ _ _ 8 _ _ 7 9
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other

numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.

- That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a **recursive object-oriented** program to find the solution to the five puzzle files.
- The output for each solution, should include:
    - The puzzle file name (e.g., puzzle1.txt)
    - The puzzle from the file printed out.
    - The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

ChatGPT Generated Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku {
private:
    char board[9][9];
    int solutionCount;

public:
    Sudoku() {
        solutionCount = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                board[row][col] = '_';
            }
        }
    }

    // Read a Sudoku puzzle from a file
    bool loadPuzzle(string filename) {
        ifstream file(filename);

        if (!file.is_open()) {
            cout << "Could not open " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (!(file >> board[row][col])) {
                    cout << "Invalid puzzle file." << endl;
                    return false;
                }

                if (board[row][col] != '_' &&
                    (board[row][col] < '1' ||
                     board[row][col] > '9')) {
```

```
                cout << "Invalid character in puzzle." << endl;
                return false;
            }
        }
    }

    file.close();
    return true;
}

// Print the current Sudoku board
void printBoard() {
    for (int row = 0; row < 9; row++) {
        if (row > 0 && row % 3 == 0) {
            cout << "------+-------+------" << endl;
        }

        for (int col = 0; col < 9; col++) {
            if (col > 0 && col % 3 == 0) {
                cout << "| ";
            }

            cout << board[row][col] << " ";
        }

        cout << endl;
    }
}

// Check whether a number can be placed in a cell
bool isValid(int row, int col, char num) {
    // Check the row
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == num) {
            return false;
        }
    }

    // Check the column
    for (int i = 0; i < 9; i++) {
```

```
        if (board[i][col] == num) {
            return false;
        }
    }

    // Check the 3x3 block
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (board[r][c] == num) {
                return false;
            }
        }
    }

    return true;
}

// Check whether the starting puzzle is valid
bool isStartingBoardValid() {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] != '_') {
                char num = board[row][col];
                board[row][col] = '_';

                bool valid = isValid(row, col, num);
                board[row][col] = num;

                if (!valid) {
                    return false;
                }
            }
        }
    }

    return true;
}
```

```
// Recursively solve the Sudoku puzzle
void solve() {
    int row = -1;
    int col = -1;

    // Find the next empty cell
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (board[r][c] == '_') {
                row = r;
                col = c;
                break;
            }
        }

        if (row != -1) {
            break;
        }
    }

    // No empty cells means a solution was found
    if (row == -1) {
        solutionCount++;

        cout << "\nSolution " << solutionCount << ":" << endl;
        printBoard();
        return;
    }

    // Try every possible digit
    for (char num = '1'; num <= '9'; num++) {
        if (isValid(row, col, num)) {
            board[row][col] = num;

            // Recursively solve the remaining puzzle
            solve();

            // Backtrack to try another possibility
            board[row][col] = '_';
```

```
            }
        }
    }

    // Display the puzzle and all solutions
    void run(string filename) {
        cout << "\n================================" << endl;
        cout << "Puzzle: " << filename << endl;
        cout << "================================" << endl;

        if (!loadPuzzle(filename)) {
            return;
        }

        cout << "\nOriginal Puzzle:\n" << endl;
        printBoard();

        solutionCount = 0;

        if (isStartingBoardValid()) {
            solve();
        }

        if (solutionCount == 0) {
            cout << "\nNo solution found" << endl;
        } else {
            cout << "\nTotal solutions: "
                 << solutionCount << endl;
        }
    }
};

int main() {
    Sudoku sudoku;

    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";
        sudoku.run(filename);
    }
```

```
    return 0;
}
```

Gemini Generated Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuBoard {
private:
    std::vector<std::vector<char>> grid;
    int solutionCount;

    // Checks if placing a digit at grid[row][col] is valid
    bool isValid(int row, int col, char num) const {
        for (int i = 0; i < 9; ++i) {
            // Check row and column
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
        }
        // Check 3x3 block
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == num) {
                    return false;
                }
            }
        }
        return true;
    }

    // Core recursive backtracking algorithm to find and print all solutions
    void solveRecursive(int row, int col) {
```

```
        // If we reach row 9, we have filled all 81 cells successfully
        if (row == 9) {
            solutionCount++;
            std::cout << "\n--- Solution #" << solutionCount << " ---\n";
            printGrid();
            return;
        }

        // Move to the next row if we finish columns in the current row
        if (col == 9) {
            solveRecursive(row + 1, 0);
            return;
        }

        // Skip cells that are already filled
        if (grid[row][col] != '_') {
            solveRecursive(row, col + 1);
            return;
        }

        // Try candidates '1' through '9' for the empty cell
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num; // Place candidate

                solveRecursive(row, col + 1); // Recurse

                grid[row][col] = '_'; // Backtrack
            }
        }
    }

public:
    SudokuBoard() : grid(9, std::vector<char>(9, '_')), solutionCount(0) {}

    // Loads the puzzle from a given file
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << "\n";
```

```
        return false;
    }

    solutionCount = 0;
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (!(file >> grid[i][j])) {
                return false;
            }
        }
    }
    return true;
}

// Prints the current state of the grid
void printGrid() const {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            std::cout << grid[i][j] << " ";
            if ((j + 1) % 3 == 0 && j < 8) std::cout << "| ";
        }
        std::cout << "\n";
        if ((i + 1) % 3 == 0 && i < 8) {
            std::cout << "---------------------\n";
        }
    }
}

// Public entry point to initiate the solver
void solve(const std::string& filename) {
    std::cout << "========================================\n";
    std::cout << "Puzzle File: " << filename << "\n";
    std::cout << "========================================\n";
    std::cout << "Original Puzzle:\n";
    printGrid();

    solveRecursive(0, 0);

    if (solutionCount == 0) {
        std::cout << "\nNo solution found\n";
```

```
        } else {
            std::cout << "\nTotal solutions found: " << solutionCount << "\n";
        }
        std::cout << "========================================\n\n";
    }
};

int main() {
    // List of puzzle files to solve
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    SudokuBoard solver;

    for (const auto& filename : puzzleFiles) {
        if (solver.loadFromFile(filename)) {
            solver.solve(filename);
        }
    }

    return 0;
}
```

Code Analysis:

Correctness: Both programs correctly use recursive backtracking to solve Sudoku puzzles and print all possible solutions. ChatGPT includes additional checks for invalid characters and conflicting starting numbers, while Gemini has fewer input validation checks.

Time Complexity: Both programs have a worst-case time complexity of O(9^k), where k is the number of empty cels. Each empty cell can have up to nine possible values, although checking Sudoku constraints reduces the number of possibilities.

Space Complexity: Both programs use O(1) space because the Sudoku board is always 9×9 and the recursion depth is limited to 81 cells. ChatGPT uses a fixed array, while Gemini uses a 2D vector.

Maintainability: Both programs are organized into classes with methods for checking numbers, printing the board, and solving puzzles. ChatGPT's code is slightly easier to follow because it separates the row, column, and block checks. Gemini's code is more compact but combines some of these checks.

Program Chosen for Assignment: ChatGPT. I chose this program because it is easier to understand and includes better input validation. It will be easier to update and maintain compared the the Gemini solution.

Improvements: I modified ChatGPT's original program to accept a puzzle filename through the command line instead of automatically solving all five puzzles. This improves maintainability because I can test indivdual puzzles and use other Sudoku files without changing the hard coded values of 1 to 5. It can also save execution time because the program only solves the puzzle I select instead of all five. The program still uses a fixed 9x9 array, so the space complexity stays the same. I also added a prologue and more detailed inline comments to explain the code and identify which parts came from ChatGPT. These changes make the program easier to use, test, and understand without changing how the Sudoku solver works.